

02_schema-event-py

#python #api

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ backend/  
    └─ app/  
        └─ schemas/  
            └─ event.py
```

El archivo `event.py` se encuentra dentro de la carpeta de esquemas del backend:

```
backend/app/schemas/
```

Este archivo define los schemas relacionados con los eventos del laboratorio SIEM MVP.

En concreto, contiene dos clases principales:

```
EventCreate  
EventOut
```

Estas clases no representan directamente tablas de PostgreSQL. Su función es definir cómo se validan los datos de entrada y cómo se devuelven los eventos desde la API.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,260p' backend/app/schemas/event.py
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`, utilizado para leer o transformar texto.

```
-n
```

Evita que `sed` imprima todo el archivo automáticamente.

```
'1,260p'
```

Indica que se impriman las líneas de la 1 a la 260.

```
backend/app/schemas/event.py
```

Es la ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```
from typing import Any, Optional  
  
from datetime import datetime
```

```

from pydantic import BaseModel, Field

class EventCreate(BaseModel):
    source: str = Field(min_length=1, max_length=64)
    severity: int = Field(ge=0, le=10)
    message: str = Field(min_length=1)

class EventOut(BaseModel):
    id: int
    ts: datetime
    source: str
    severity: int
    message: str
    meta: Optional[dict[str, Any]] = None
    created_at: datetime

    model_config = {"from_attributes": True}

```

4 Función general del archivo

El archivo `schemas/event.py` define los schemas relacionados con eventos.

Estos schemas cumplen dos funciones distintas:

```

EventCreate → valida los datos necesarios para crear un evento simple.
EventOut    → define cómo se devuelve un evento desde la API.

```

El schema `EventCreate` se utiliza en el endpoint:

```
POST /events
```

Dentro de:

```
backend/app/api/routes/events.py
```

Aparece así:

```
def create_event(payload: EventCreate, db: Session = Depends(get_db)):
```

Esto significa que FastAPI validará el cuerpo de la petición usando `EventCreate`.

El schema `EventOut` se utiliza como modelo de respuesta:

```
@router.post("", response_model=EventOut)
```

y también:

```
@router.get("", response_model=list[EventOut])
```

Esto indica que la API devolverá eventos con la estructura definida por `EventOut`.

5 Estructura general del archivo

El archivo puede dividirse en cuatro bloques:

```
from typing import Any, Optional
```

Importación de tipos para campos flexibles y opcionales.

```
from datetime import datetime
from pydantic import BaseModel, Field
```

Importación de fechas y herramientas de Pydantic.

```
class EventCreate(BaseModel):
    ...
```

Schema para crear eventos.

```
class EventOut(BaseModel):
    ...
```

Schema para devolver eventos desde la API.

Visualmente:

```
event.py
├─ Importaciones typing
├─ Importación datetime
├─ Importaciones Pydantic
├─ EventCreate
│   ├── source
│   ├── severity
│   └── message
└─ EventOut
    ├── id
    ├── ts
    ├── source
    ├── severity
    ├── message
    ├── meta
    ├── created_at
    └── model_config
```

6 Análisis línea por línea

Importación de Any y Optional

```
from typing import Any, Optional
```

Esta línea importa dos tipos desde el módulo estándar `typing`.

Any

```
Any
```

Representa cualquier tipo de dato.

En este archivo se usa dentro del campo:

```
meta: Optional[dict[str, Any]] = None
```

Esto permite que el diccionario `meta` tenga claves de tipo `str` y valores de cualquier tipo.

Ejemplo:

```
{
    "host": "server-01",
```

```
"ip": "192.168.1.10",
"attempts": 5,
"blocked": true
}
```

Los valores pueden ser texto, números, booleanos u otros tipos compatibles con JSON.

Optional

Optional

Indica que un campo puede tener un valor del tipo indicado o puede ser `None`.

En este archivo se usa para:

```
meta: Optional[dict[str, Any]] = None
```

Esto significa que `meta` puede ser un diccionario o puede no existir.

Importación de `datetime`

```
from datetime import datetime
```

Esta línea importa `datetime` desde el módulo estándar `datetime`.

Se utiliza en el schema `EventOut` para campos de fecha y hora:

```
ts: datetime
created_at: datetime
```

Estos campos proceden del modelo SQLAlchemy `Event`.

En la respuesta JSON, FastAPI y Pydantic transforman estos objetos `datetime` en cadenas de texto en formato compatible con JSON.

Ejemplo:

```
2026-01-15T16:07:14.286702
```

Importación de `BaseModel` y `Field`

```
from pydantic import BaseModel, Field
```

Esta línea importa dos elementos desde Pydantic:

```
BaseModel
Field
```

`BaseModel`

`BaseModel` es la clase base de Pydantic.

Las clases que heredan de `BaseModel` se convierten en schemas de validación y serialización.

En este archivo se usa en:

```
class EventCreate(BaseModel):
```

y:

```
class EventOut(BaseModel):
```

Field

`Field` permite definir restricciones sobre los campos.

Se usa en `EventCreate` para validar:

```
source
severity
message
```

Por ejemplo:

```
severity: int = Field(ge=0, le=10)
```

indica que la severidad debe ser un entero entre 0 y 10.

Definición de la clase `EventCreate`

```
class EventCreate(BaseModel):
```

Esta línea define el schema `EventCreate`.

Desglose:

```
class
```

Palabra clave de Python para definir una clase.

```
EventCreate
```

Nombre de la clase.

El nombre indica que este schema se usa para crear eventos.

```
(BaseModel)
```

Indica que hereda de `BaseModel`.

Por tanto, Pydantic validará automáticamente los datos que usen este schema.

Este schema se usa en:

```
def create_event(payload: EventCreate, db: Session = Depends(get_db)):
```

dentro de `events.py`.

Campo `source` en `EventCreate`

```
source: str = Field(min_length=1, max_length=64)
```

Define el campo `source`.

Representa el origen del evento.

Desglose:

```
source
```

Nombre del campo.

```
: str
```

Debe ser una cadena de texto.

```
Field(min_length=1, max_length=64)
```

Define restricciones de longitud.

Restricción `min_length=1`

```
min_length=1
```

Impide que el origen del evento esté vacío.

Ejemplo inválido:

```
{  
  "source": ""  
}
```

Restricción `max_length=64`

```
max_length=64
```

Impide que `source` supere 64 caracteres.

Esto está alineado con el modelo `Event`, donde la columna `source` está definida como:

```
String(64)
```

Así se valida antes de llegar a PostgreSQL.

Campo `severity` en `EventCreate`

```
severity: int = Field(ge=0, le=10)
```

Define el campo `severity`.

Representa la severidad del evento.

Desglose:

```
severity
```

Nombre del campo.

```
: int
```

Debe ser un número entero.

```
Field(ge=0, le=10)
```

Define el rango permitido.

Restricción `ge=0`

```
ge=0
```

Significa `greater or equal`.

La severidad debe ser mayor o igual que 0.

Restricción `le=10`

```
le=10
```

Significa `less or equal`.

La severidad debe ser menor o igual que 10.

Ejemplos válidos:

```
0
5
10
```

Ejemplos inválidos:

```
-1
11
```

Campo `message` en `EventCreate`

```
message: str = Field(min_length=1)
```

Define el campo `message`.

Representa el mensaje descriptivo del evento.

Desglose:

```
message
```

Nombre del campo.

```
: str
```

Debe ser una cadena de texto.

```
Field(min_length=1)
```

Exige al menos un carácter.

Esto evita crear eventos sin mensaje.

Ejemplo inválido:

```
{
  "source": "auth",
  "severity": 3,
  "message": ""
}
```

Definición de la clase `EventOut`

```
class EventOut(BaseModel):
```

Esta línea define el schema `EventOut` .

Este schema se usa para devolver eventos desde la API.

A diferencia de `EventCreate` , que define datos de entrada, `EventOut` define datos de salida.

Se utiliza como `response_model` en endpoints como:

```
@router.post("", response_model=EventOut)
```

y:

```
@router.get("", response_model=list[EventOut])
```

Esto permite que FastAPI devuelva únicamente los campos definidos en `EventOut` .

Campo `id`

```
id: int
```

Define el campo `id` .

Representa el identificador único del evento en la base de datos.

Este campo no aparece en `EventCreate` , porque el cliente no lo envía.

Lo genera la base de datos al insertar el registro.

Relación:

```
POST /events recibe EventCreate
      ↓
PostgreSQL asigna id
      ↓
API devuelve EventOut con id
```

Campo `ts`

```
ts: datetime
```

Define el campo `ts` .

Representa el timestamp asociado al evento.

En el modelo `Event` , este campo está definido como:

```
ts: Mapped[object] = mapped_column(DateTime(timezone=True), server_default=func.now(), nullable=False)
```

En la respuesta, Pydantic lo trata como un `datetime` .

Campo `source`

```
source: str
```

Define el campo `source` en la respuesta.

Representa el origen del evento.

Ejemplo:


```
{
  "source": "auth"
}
```

Campo severity

```
severity: int
```

Define el campo `severity` en la respuesta.

Representa la severidad del evento.

Ejemplo:

```
{
  "severity": 4
}
```

Campo message

```
message: str
```

Define el campo `message` en la respuesta.

Representa el mensaje descriptivo del evento.

Ejemplo:

```
{
  "message": "Failed login attempt for user admin"
}
```

Campo meta

```
meta: Optional[dict[str, Any]] = None
```

Define el campo `meta` en la respuesta.

Este campo puede contener metadatos adicionales del evento o puede ser `None`.

Desglose:

```
meta
```

Nombre del campo.

```
Optional[dict[str, Any]]
```

Puede ser un diccionario o `None`.

```
-
```

Valor por defecto si no existe.

Este campo está alineado con el modelo `Event`, donde `meta` se guarda como `JSONB`.

Campo created_at

```
created_at: datetime
```

Define el campo `created_at`.

Representa la fecha y hora en la que el evento fue creado en la base de datos.

En el modelo `Event`, se define con:

```
server_default=func.now()
```

Por tanto, normalmente PostgreSQL asigna este valor automáticamente.

Separación antes de `model_config`

Hay una línea en blanco antes de `model_config`.

No afecta al funcionamiento.

Solo separa visualmente los campos del schema de su configuración.

Configuración `model_config`

```
model_config = {"from_attributes": True}
```

Esta línea configura el comportamiento de Pydantic.

En Pydantic v2, `from_attributes=True` permite crear el modelo de salida a partir de atributos de un objeto.

Esto es importante porque los endpoints devuelven objetos SQLAlchemy, no diccionarios puros.

Por ejemplo, en `events.py`:

```
return ev
```

`ev` es un objeto `Event` de SQLAlchemy.

Gracias a:

```
model_config = {"from_attributes": True}
```

Pydantic puede leer atributos como:

```
ev.id
ev.ts
ev.source
ev.severity
ev.message
ev.meta
ev.created_at
```

y convertirlos en una respuesta JSON con estructura `EventOut`.

Sin esta configuración, Pydantic podría no saber cómo transformar correctamente el objeto ORM.

Resultado final del archivo

Después de cargar este archivo, quedan disponibles dos schemas:

```
EventCreate
EventOut
```

Resumen:

```
EventCreate
├── source
├── severity
└── message

EventOut
├── id
├── ts
├── source
├── severity
├── message
├── meta
└── created_at
```

`EventCreate` se usa para entrada.

`EventOut` se usa para salida.

7 Relación con el flujo técnico del laboratorio

Este archivo participa en la validación y serialización de eventos.

Flujo de creación simple desde `/events` :

```
Cliente envía JSON
    ↓
FastAPI valida con EventCreate
    ↓
se crea Event
    ↓
se guarda en PostgreSQL
    ↓
se devuelve con EventOut
```

Flujo de consulta desde `/events` :

```
Cliente solicita GET /events
    ↓
se consultan objetos Event
    ↓
Pydantic aplica EventOut
    ↓
se devuelve lista JSON
```

Relación con `/ingest` :

```
POST /ingest
    ↓
crea Event
    ↓
devuelve EventOut
```

Aunque `/ingest` usa `IngestPayload` como entrada, también usa `EventOut` como respuesta.

8 Errores típicos o puntos importantes

`EventCreate` no incluye `meta`

El schema `EventCreate` solo incluye:

```
source
severity
```

```
message
```

Por tanto, el endpoint `POST /events` no está pensado para crear eventos con metadatos.

Para enviar `meta`, se usa mejor:

```
POST /ingest
```

con `IngestPayload`.

EventOut sí incluye meta

Aunque `EventCreate` no incluye `meta`, `EventOut` sí lo devuelve.

Esto permite que eventos creados por `/ingest`, que sí pueden tener `meta`, aparezcan correctamente al consultar `/events`.

id, ts y created_at no se envían al crear

Estos campos aparecen en `EventOut`, pero no en `EventCreate`.

Motivo:

```
id          → lo genera PostgreSQL
ts          → lo genera el backend o la base de datos
created_at → lo genera PostgreSQL
```

El cliente no necesita enviarlos.

Las validaciones protegen la base de datos

Las restricciones de `EventCreate` evitan errores antes de insertar datos:

```
source → longitud compatible con String(64)
severity → rango controlado
message → no vacío
```

Esto reduce errores en PostgreSQL.

from_attributes es necesario para devolver modelos ORM

Como los endpoints devuelven objetos SQLAlchemy, `EventOut` necesita:

```
model_config = {"from_attributes": True}
```

para poder convertir objetos ORM en respuestas JSON.

9 Comandos útiles relacionados

Probar creación de evento simple con `/events`:

```
curl -X POST http://localhost:8000/events \
-H "Content-Type: application/json" \
-d '{
  "source": "auth",
  "severity": 3,
  "message": "Simple event created from /events"
}'
```

Listar eventos:

```
curl http://localhost:8000/events
```

Listar eventos con límite:

```
curl "http://localhost:8000/events?limit=10"
```

Probar error por severity inválida:

```
curl -X POST http://localhost:8000/events \
-H "Content-Type: application/json" \
-d '{
  "source": "auth",
  "severity": 99,
  "message": "Invalid severity"
}'
```

Probar que el schema se puede importar:

```
docker exec -it siem-api python -c "from app.schemas.event import EventCreate, EventOut;
print(EventCreate, EventOut)"
```

Comprobar Swagger:

```
http://localhost:8000/docs
```

Siguiente nota:

```
```text
04_API-de-ingesta
└─ 05_Analisis-tecnico-de-ingesta
 └─ 03_ingest-py
```